

# Copying Symlinked Directories Recursively - Google Gemini

Gemini - August 16, 2026 at 08:58 PM

YOU

How do we use cp to copy those direct directories symlinked into so as to get that wordlists data

GEMINI

To follow symbolic links and copy the actual files and directories targeted by the symlinks (rather than trying to recreate the symlinks themselves on the shared folder), use the `-L` (dereference) flag with `cp`. Run the following command:

Bash

```
cp -rvL /usr/share/wordlists/ /media/sf_vm/
```

Key Flags Breakdown:

- `-L` (`--dereference`): Forces `cp` to follow symbolic links and copy the actual files/directories they point to. This resolves the Operation not permitted error caused by VirtualBox shared folders (`/media/sf_vm/`) blocking the creation of symlinks.
- `-r` or `-R` (`--recursive`): Copies directories recursively, ensuring contents inside symlinked folders (like `/usr/share/seclists`) get copied.
- `-v` (`--verbose`): Prints the progress to show what is being copied.

YOU

Error writing protocol error, what are those about

GEMINI

That error in your terminal reading:

Plaintext

rsync error: some files/attrs were not transferred (see previous errors) (code 23) at main.c(1338)  
occurs when rsync attempts to transfer file metadata or symlinks that the target file system cannot accept.

What Caused It

VirtualBox Shared Folder Limitations: The destination /media/sf\_vm/ is a VirtualBox Shared Folder mounted from a Windows/host filesystem (such as NTFS or exFAT). These filesystems do not support standard Linux permissions, ownership attributes (chown/chgrp), or symbolic link creation.

Unsupported Symlinks & Metadata: When rsync tried to copy symlinks (like amass, dirb, seclists, etc.) and preserve their attributes, the underlying VirtualBox driver rejected the operation, throwing the code 23 partial transfer error.

How to Fix It with rsync

To perform this transfer smoothly using rsync instead of cp, add the -L (copy referenced files instead of symlinks) flag and disable attribute preservation (--no-perms --no-owner --no-group):

Bash

```
rsync -rvL --no-perms --no-owner --no-group /usr/share/wordlists/ /media/sf_vm/wordlists/
```

-L (--copy-links): Follows symlinks and copies the actual target files/folders instead of creating broken symlinks on the shared drive.

--no-perms --no-owner --no-group: Prevents rsync from trying to set Linux permissions and ownership on the host filesystem, preventing Operation not permitted errors.

GEMINI

The new error visible in your screenshot:

Plaintext

```
rsync: [receiver] write failed on "/media/sf_vm/wordlists/...": ...
```

```
rsync error: error in file IO (code 11) at receiver.c(380)
```

```
rsync: [sender] write error: Broken pipe (32)
```

indicates an I/O error (Input/Output write failure). This happens almost exclusively for three reasons when writing to a VirtualBox shared folder:

Root Causes

**Storage Space Exhausted:** Your host OS drive (where /media/sf\_vm/ lives) or your VM drive ran out of disk space. SecLists and wordlists can take up several gigabytes.

**File Size Limit (FAT32):** If your host drive or USB drive is formatted as FAT32, it cannot handle individual files larger than 4 GB or directories with high file counts.

**VirtualBox Shared Folder Cache / Transport Glitch:** High-volume file transfers over vboxfs can cause the shared folder driver process to freeze or drop the connection (causing the Broken pipe error).

How to Fix It

#### 1. Check Available Disk Space

Check space on both your Linux guest and mounted shared folder:

Bash

```
df -h /media/sf_vm/
```

If Use% is at 100%, free up space on your host operating system drive.

#### 2. Archive First (Recommended Bypass)

Instead of transferring hundreds of thousands of individual files directly through VirtualBox's shared folder interface, compress the files into an archive on your VM first, then copy the single .tar.gz file over:

Bash

# 1. Compress the wordlists directory following symlinks

```
tar -czvhf /tmp/wordlists.tar.gz /usr/share/wordlists/
```

# 2. Copy the single archive file to the shared folder

```
cp /tmp/wordlists.tar.gz /media/sf_vm/
```

-h (--dereference): Tells tar to follow symlinks and archive the actual files.

Once moved to the host OS, you can extract wordlists.tar.gz directly on your host machine. This eliminates VirtualBox's file I/O bottleneck and avoids broken pipe errors.

YOU

I usually have 16gb of swap space partition. Is it possible to use that partition to store some data like 4g since I am out of storage and still be capable as being used for swap space during hibernation since i don't hibernate with vast apps open. And I don't wanna repartition drives this moment

GEMINI

No, you cannot store regular files directly on an active swap partition without standard repartitioning or reformatting, because swap space lacks a file system (like ext4, NTFS, or FAT32).

Swap space is raw block storage used by the Linux kernel to dump raw memory pages. Writing files to that raw partition would corrupt the swap space (and break hibernation), while enabling swap on it would wipe any raw data you attempted to store there.

However, since you explicitly want to avoid repartitioning or reformatting your drives, here are the two safest ways to safely shrink your swap usage and reclaim space for your files:

#### Method 1: Convert Swap to a File & Reclaim Disk Space (No Repartitioning)

If your swap space is currently set up as a swap file (common on modern Linux distributions) rather than a fixed dedicated disk partition, you can simply shrink it to 12 GB, instantly giving you 4 GB of regular disk space back.

Check if you are using a swap file or partition:

Bash

```
swapon --show
```

If the NAME output shows a path like /swapfile:

Turn off swap temporarily:

Bash

```
sudo swapoff -a
```

Resize the swap file to 12 GB:

Bash

```
sudo fallocate -l 12G /swapfile
```

# or using dd if fallocate isn't supported on your filesystem:

```
# sudo dd if=/dev/zero of=/swapfile bs=1G count=12 mode=600
```

Re-initialize and re-enable swap:

Bash

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

This immediately leaves 4 GB of free space on your root drive for normal file storage without touching partition tables.

#### Method 2: Temporary Loopback Image (If Swap is a Raw Partition)

If your swap is a hard partition (e.g., /dev/sda2 or /dev/nvme0n1p2) and you refuse to modify disk partitions right now, you can turn off swap, create a temporary 4 GB ext4 image inside it, mount it as a folder, and leave the remaining 12 GB for swap.

Disable the swap partition:

Bash

```
sudo swapoff /dev/sdXn # Replace with your actual swap partition name
```

Format 4 GB of the partition into an ext4 loop filesystem:

Bash

```
# Create a 4GB filesystem starting at offset 0
```

```
sudo mkfs.ext4 -b 4096 /dev/sdXn 1048576
```

```
# Mount it to access your 4GB of storage
```

```
sudo mkdir -p /mnt/temp_storage
```

```
sudo mount /dev/sdXn /mnt/temp_storage
```

```
sudo chown -R $USER:$USER /mnt/temp_storage
```

Use the remaining 12 GB as swap:

Bash

```
# Initialize swap starting after the 4GB mark (offset 4194304 KB)
```

```
sudo mkswap --offset 4194304 /dev/sdXn
```

```
sudo swapon /dev/sdXn
```

⚠ Warning on Method 2: While mathematically possible, using offset swap on a shared raw partition is risky. If system scripts auto-run standard `mkswap /dev/sdXn` on reboot, your 4 GB of stored files will be overwritten.

#### Summary Recommendation

If you are running low on disk space while transferring large files (like SecLists or rockyou.txt), the most robust solution without touching partitions is to replace your partition swap with a flexible 12 GB Swap File on your primary partition. This safely gives you 4 GB of standard storage back without risking hibernation integrity or unformatted data loss.